



UNIVERSITÀ DI PARMA

Image Formats

- Image file types
- Key terms for images
 - Pixels, pixel depth
- Main image formats
- Image files
 - Raster Formats: PNM, BMP, GIF, PNG, JPEG
 - Vector Formats: SVG
- (Video files)





Raster Format

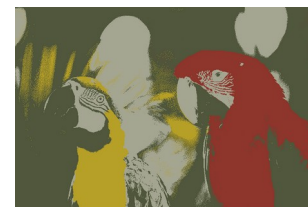


Raster Format

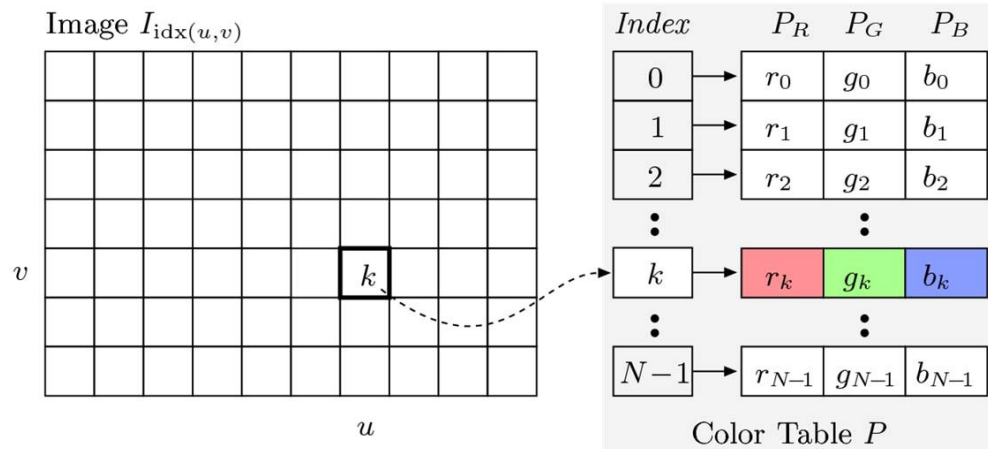


- Also known as Bitmap format
 - Dot matrix data, namely a rectangular grid of **pixels**
 - Each pixel encodes “dot” information
 - Fixed resolution
 - Density $\rightarrow$ DPI
 - Widely used for digital cameras, scanners, printing...
 - A lot of different file formats

- Smallest *addressable* image element
 - Can be seen as a sample of the original image
- A pixel can contain different information
 - For greyscale images: luminance
 - Each pixel encodes “grey level”
 - Number of levels depends on **pixel depth**
 - Often 256 levels (0→black, 255→white), namely 8 bits
 - 2 levels for printing (monochrome, only 1 bit for pixel!)
 - Color
 - Again different levels → different number of colors
 - Also different formats



- Paletted images
 - Indexed colors
 - Also known as *color lookup table*



Credits: Emmanuel Agu

1972×1600

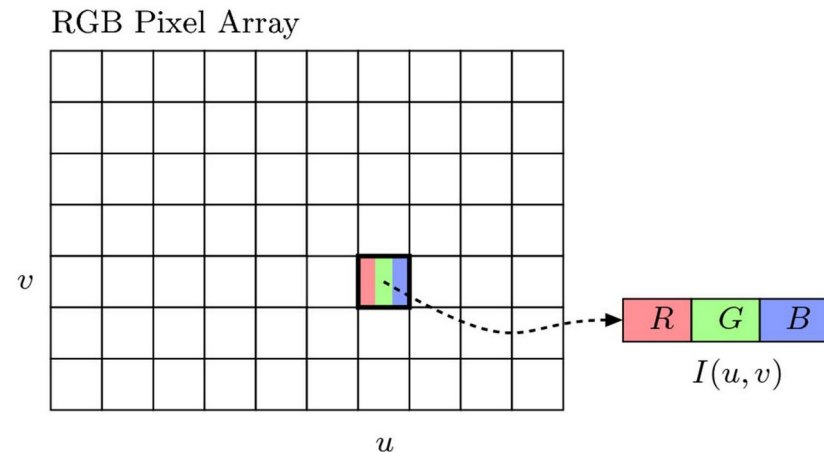


RGB 24 bit
~9 Mbytes

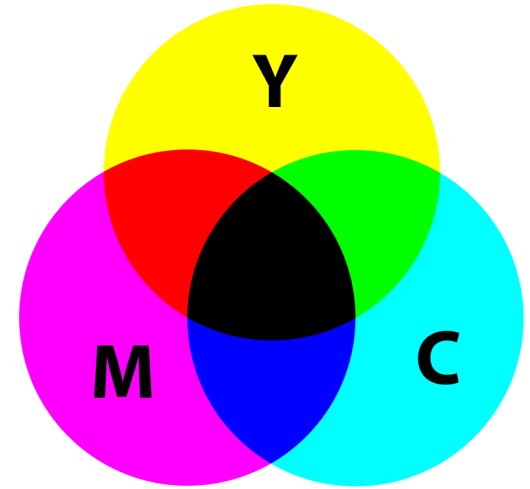


Palette 256 colors
~3 Mbytes

- RGB
 - Each pixel encodes Red, Green, Blue channels
 - Different depths (24 bit considered as “true color”)
 - OpenCV uses BGR as default
- RGBA
 - Alpha channel used as % for transparency



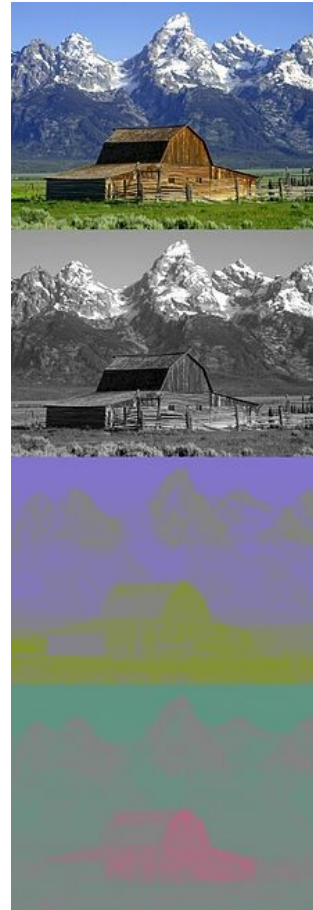
- Cyan, Magenta, Yellow and black (Key)
- Subtractive color model
 - Cyan + Magenta + Yellow = black
- Main usage: printed materials



- Y'UV and Y'C_bC_r
 - Trying to mimic human vision
 - Cones & Rods cells
 - Luminance has a bigger importance than **Chrominance**
 - Do not encode colors but colors difference...
 - Widely used for image/video compression

$$\begin{pmatrix} Y \\ U \\ V \end{pmatrix} = \begin{pmatrix} 0.299 & 0.587 & 0.114 \\ -0.147 & -0.289 & 0.436 \\ 0.615 & -0.515 & -0.100 \end{pmatrix} \cdot \begin{pmatrix} R \\ G \\ B \end{pmatrix} \quad \begin{pmatrix} R \\ G \\ B \end{pmatrix} = \begin{pmatrix} 1.000 & 0.000 & 1.140 \\ 1.000 & -0.395 & -0.581 \\ 1.000 & 2.032 & 0.000 \end{pmatrix} \cdot \begin{pmatrix} Y \\ U \\ V \end{pmatrix}$$

$$Y = 0.299R + 0.587G + 0.114B, U = 0.492(B-Y), V = 0.877(R-Y)$$



- Standardized variant of YUV
- Used for digital videos and some image format

$$Y = w_R \cdot R + (1 - w_B - w_R) \cdot G + w_B \cdot B$$

$$C_b = \frac{0.5}{1 - w_B} \cdot (B - Y)$$

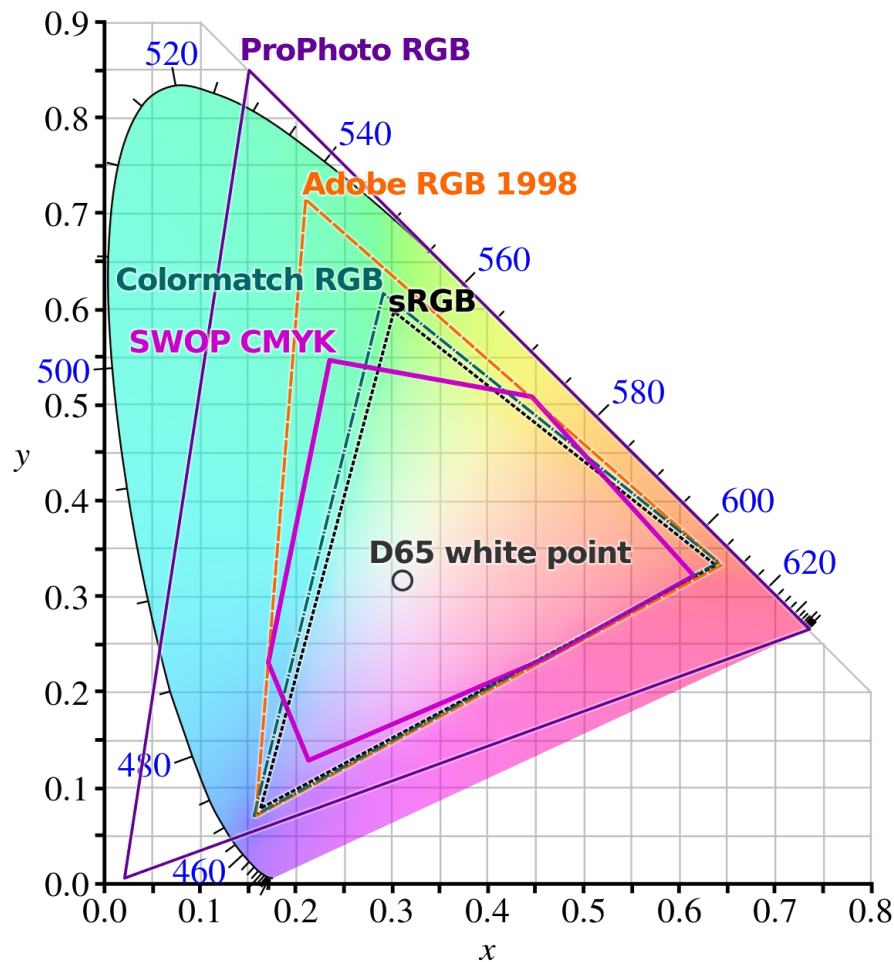
$$C_r = \frac{0.5}{1 - w_R} \cdot (R - Y)$$

$$R = Y + \frac{1 - w_R}{0.5} \cdot C_r$$

$$G = Y - \frac{w_B \cdot (1 - w_B) \cdot C_b - w_R \cdot (1 - w_R) \cdot C_r}{0.5 \cdot (1 - w_B - w_R)}$$

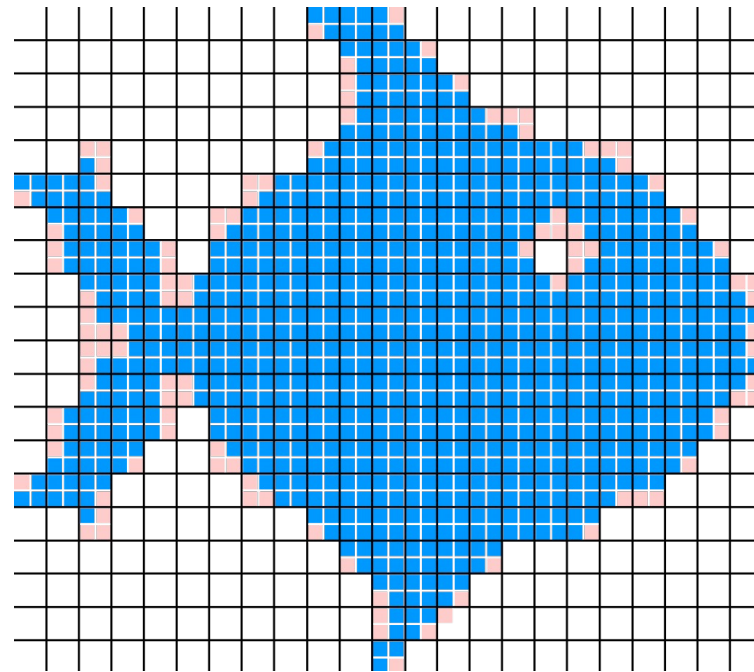
$$B = Y + \frac{1 - w_B}{0.5} \cdot C_b$$

- Subset of colors that can be represented



Raster (bitmap) format

- For raster images, usually:
 - Header:
 - Format
 - Size
 - Color
 - Compression
 - ...
 - Bitmap
 - Pixel array(s)
- Every image standard features its own header/bitmap format



- Portable Bit/Grey/Pix Map Format
 - .pbm, .pgm, .ppm
- Uncompressed format
- Very easy to parse
 - In some cases an ASCII editor can be used
- Born in the UNIX Environment
- ASCII Header
 - Format, Size, Comments
 - Comments widely used to encode additional info
- RAW or ASCII bitmap
- Video extension PVM

PNM Header & Bitmap

P4 = PBM
P5 = PGM
P6 = PPM

P5
320 240
some free text
255

Image Size

Comments (add info)

Maximum channel value



PBM, PGM: 1 byte/pixel
PPM: 3 bytes/pixel

- As memory bitmap

0	0	0	0	0	0	0	0
0	0	255	255	255	0	0	0
0	0	127	255	127	0	60	0
0	0	0	255	127	127	60	0
0	0	0	0	0	70	60	0
0	0	0	0	0	255	255	255

0	0	0	0	0	0	0	0
0	0	25	25	25	0	0	0
		5	5	5			
0	0	12	25	12	0	60	0
		7	5	7			
0	0	0	25	12	12	60	0
			5	7	7		
0	0	0	0	0	70	60	0
0	0	0	0	0	25	25	25
					5	5	5

- The more the information the bigger the memory required
- 4k images
 - $3840 \times 2160 \rightarrow 24 \text{ bit color} \rightarrow 23 \text{ Mbytes}$
- Size affects:
 - Memory (this can not be an issue for us!)
 - Storage
 - Transmission

- Compression can be mandatory
- Two different policies
- Lossless
 - Original info is preserved
 - Better choice for machine vision
 - TIFF, BMP, PNG, GIF, JPEG
- Lossy
 - Some info is lost
 - JPEG

- Adobe Developers Association,
 - TIFF (TM) Revision 6.0 - Final, June 3, 1992
 - Compressed, lossless
 - Compression:
 - None, ZIP, LZW ...
 - Variable color depth
 - Multiple pages (Fax)

- Device Independent Bitmap (DIB)
 - Typical extensions are .bmb or, less frequently, .dib
- File structure
 - BITMAPFILEHEADER bmfh;
 - type, size & layout (pixel ↔ lenght)
 - BITMAPINFOHEADER bmih;
 - size, compression and color format
 - RGBQUAD aColors[];
 - Color palette
 - Not present for 24 bit format (24-bit red-green-blue (RGB) for each pixel)
 - Colors are ordered on frequency basis (dithering)
 - BYTE aBitmapBits[];
 - Index or intensity of colors
 - Run-length encoded (RLE) compression

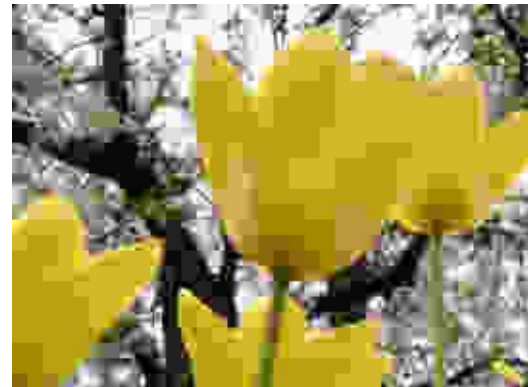
- Used for web and small animations
- Based on a color palette
 - 8 bit
 - Maximum 256 colors on 16 M max
 - Transparency support
- Patented compression algorithm
 - Big issue

- PNG coped with patent issues
 - Portable Network Graphic
 - 24 bit depth (~16 millions of colors) + Alpha channel
- “Free” compression algorithm
- No animation support! → this explains why GIFs are still alive..
- 8 bytes initial header (magic number)
 - 89 50 4E 47 0D 0A 1A 0A (as ASCII first four are “.PNG”)
- Other “headers” and data are following as chunks
 - 4 bytes chunk length
 - 4 bytes chunk type (IHDR header, PLTE optional palette, IDAT actual data, IEND end of file)
 - Chunk data
 - 4 bytes CRC

- JPEG: Joint Photographic Experts Group
- Born in the '80s (standard in '90s)
- JPEG is also a compression algorithm
- The idea was to have a platform independent (PC, Mac ...) file format
- Can use RGB, CMYK, YUV (maybe others)
- Magic number FF D8 FF

- Compression based on luminance and chrominance
- RGB o CMYK values are converted in $Y'C_bC_r$
 - Luminance + Chrominance
- Y and C_x are separately compressed
- For our eyes luminance is more important than colors
 - If a lossy compression is used, Luminance is better preserved
- Compression
 - Image is subdivided in 8x8 pixels blocks
 - Discrete Cosine Transform (~Fourier) is applied to each block
 - Results are quantized
 - Based on assumption the our eye is more sensitive to low frequency changes
 - Basically this wipes our small details

- Different compression levels can be used depending on image usage
 - Up to 20 → internet
 - Up to 5 → printed matter
- Good results with images without abrupt changes, namely “low frequency” images
- Bad if sharp color changes
- Avoid, if possible, for machine vision



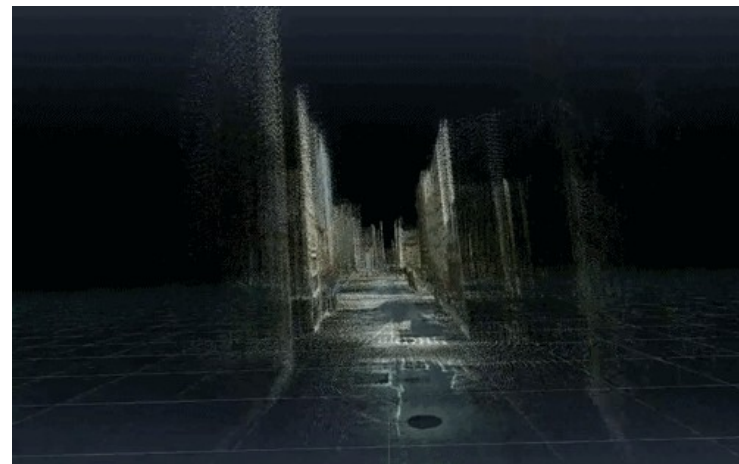
- Vector Graphics
- Described using a language!
 - Based on XML W3
- Different objects
 - Polylines, namely a combination of 1 or more straight and curve lines
 - This allows to obtain all shapes
 - Raster images
 - Text
 - ...

SVG: example

```
<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<!-- Created with Inkscape (http://www.inkscape.org/) -->
<svg
  xmlns:dc="http://purl.org/dc/elements/1.1/"
  xmlns:cc="http://creativecommons.org/ns#"
  xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
  xmlns:svg="http://www.w3.org/2000/svg"
  xmlns="http://www.w3.org/2000/svg"
  xmlns:sodipodi="http://sodipodi.sourceforge.net/DTD/sodipodi-0.dtd"
  xmlns:inkscape="http://www.inkscape.org/namespaces/inkscape"
  width="1052.3622"
  height="744.09448"
  id="svg2"
  version="1.1"
  inkscape:version="0.48.1 r9760"
  sodipodi:docname="Nuovo documento 1">
  <defs
    id="defs4" />
  <sodipodi:namedview
    id="base"
    pagecolor="#ffffff"
    bordercolor="#666666"
    borderopacity="1.0"
    inkscape:pageopacity="0.0"
    inkscape:randomized="0"
    d="m 251.42858,175.52305 -61.11845,4.77394 -30.71632,53.05434
-23.42693,-56.65186 -59.949534,-12.81827 46.639804,-39.78672 -6.33453,-
60.976461 52.25191,32.062318 56.03459,-24.867261 -14.34635,59.602324 z"
    transform="translate(0,308.2677)"
    inkscape:transform-center-x="1.8913327"
    inkscape:transform-center-y="6.9973399" />
  </g>
</svg>
```



- Not images!
- Set of **data** points in the 3D space
- Free format
 - ASCII is often used
- We will use them sometime



Which format to use for Machine Vision?

- Not always possible to choose
 - Avoid lossy formats
 - Limited depth formats
 - Quick & dirt approach → maybe you will have to purchase additional storage
- Not always necessary to choose
 - OpenCV
 - GOLD

- For processing it is “necessary” to load images
- Bidimensional array seems the straightforward choice
 - But monodimensional structures are widely used
- Carefully consider pixel data
- We will investigate the OpenCV approaches



UNIVERSITÀ DI PARMA

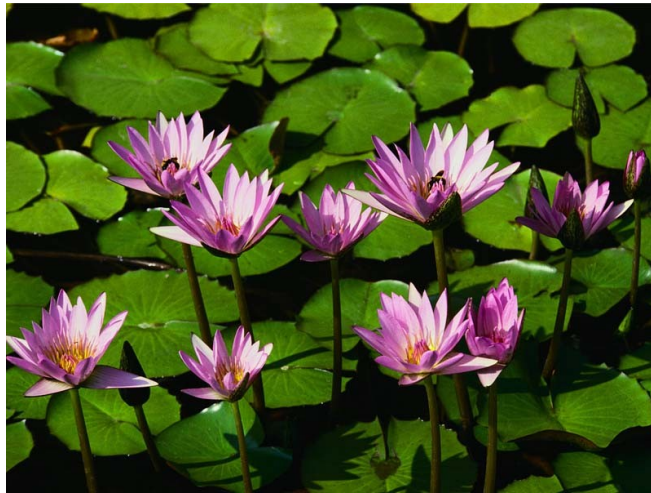
Image Formats

Question time!



- Image file types
- Key terms for images
 - Pixels, pixel depth
- Main image formats
- Image files
 - Raster Formats: PNM, BMP, GIF, PNG, JPEG
 - Vector Formats: SVG
- (Video files)

In queste slide impareremo i concetti base delle immagini e anche come sono fatti comuni formati immagine su file



Partiamo con il formato raster, che è comunque quello su cui ci focalizzeremo, e ingrandiamo progressivamente l'immagine.







Si iniziano a vedere de quadratini, cosa sono?

- Also known as Bitmap format
 - Dot matrix data, namely a rectangular grid of **pixels**
 - Each pixel encodes “dot” information
 - Fixed resolution
 - Density → DPI
 - Widely used for digital cameras, scanners, printing...
 - A lot of different file formats

In pratica l'immagine è una **matrice** di punti (che si chiamano pixel). Quindi è una struttura dati che fondamentalmente posso organizzare come matrice. E ogni punto cosa contiene? Beh dipende un po' dal tipo di immagine ma di fatto contiene le informazioni che descrivono il punto stesso. Nell'immagine in esempio conterrà il colore del pixel stesso (tipicamente, ma esistono tante altre possibilità, i valori dei canali R – rosso, G – verde e B – blu.

La matrice chiaramente è caratterizzata da numero di righe e numero di colonne

DPI → dot per inches

Poi un conto è come è fatta l'informazione che devo avere e un conto è come la metto in un file



- Smallest *addressable* image element
 - Can be seen as a sample of the original image
- A pixel can contain different information
 - For greyscale images: luminance
 - Each pixel encodes “grey level”
 - Number of levels depends on **pixel depth**
 - Often 256 levels (0→black, 255→white), namely 8 bits
 - 2 levels for printing (monochrome, only 1 bit for pixel!)
 - Color
 - Again different levels → different number of colors
 - Also different formats



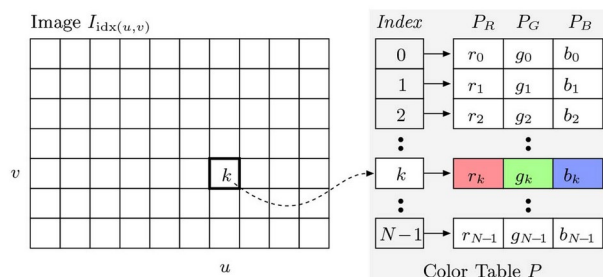
24 bit

8 bit

6 bit

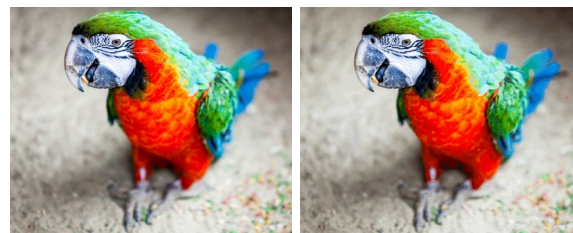
2 bit

- Paletted images
 - Indexed colors
 - Also known as *color lookup table*



Credits: Emmanuel Agu

1972×1600



RGB 24 bit
~9 Mbytes

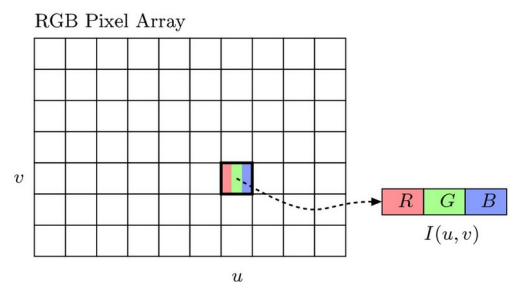
Palette 256 colors
~3 Mbytes

Spesso per memorizzare i colori si usa una tabella di appoggio che contiene i solo colori usati. Nella matrice immagine trovo l'indice della tabella che contiene i valori necessari per la rappresentazione dello specifico colore (ad esempio RGB).

È un indirizzamento indiretto. Perché questa complicazione? Perché mi fa risparmiare spazio. Molto usata per i formati immagine più vetusti (ma ancora in uso oggi come GIF) e soprattutto si limita il numero massimo di colori usabile (attenzione, non l'intervallo)

L'immagine del pappagallo in pratica in formato RGB occupa 3 volte tanto (3 byte per pixel), mentre in formato palettizzato si usa 1 byte per pixel e poi 3*256 per la palette (in questo caso potevo usare una palette più ampia senza aumentare di troppo lo spazio)

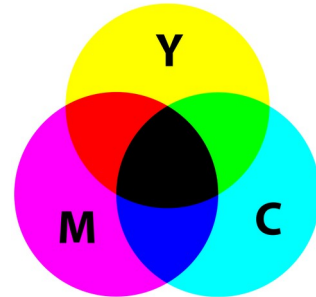
- RGB
 - Each pixel encodes Red, Green, Blue channels
 - Different depths (24 bit considered as “true color”)
 - OpenCV uses BGR as default
- RGBA
 - Alpha channel used as % for transparency



La rappresentazione è quella “packed” perché nella unpacked i canali sono array separati

Attenzione che i colori rappresentabili con il modello RGB comunque hanno una estensione (GAMUT) a quella del visibile

- Cyan, Magenta, Yellow and black (Key)
- Subtractive color model
 - Cyan + Magenta + Yellow = black
- Main usage: printed materials

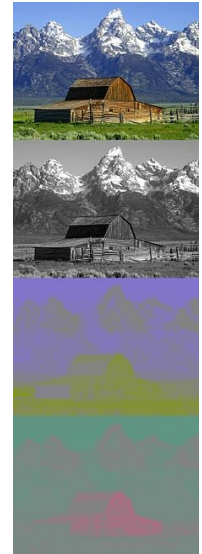


Riflette solo una parte quindi “sottrae”
Tecnicamente ne basterebbero 3 ma di fatto ci vuole anche il nero in quanto i pigmenti non sono perfetti e poi si sovrappongono

- Y'UV and Y'C_bC_r
 - Trying to mymic human vision
 - Cones & Rods cells
 - Luminance has a bigger importance than **Chrominance**
 - Do not encode colors but colors difference...
 - Widely used for image/video compression

$$\begin{pmatrix} Y \\ U \\ V \end{pmatrix} = \begin{pmatrix} 0.299 & 0.587 & 0.114 \\ -0.147 & -0.289 & 0.436 \\ 0.615 & -0.515 & -0.100 \end{pmatrix} \cdot \begin{pmatrix} R \\ G \\ B \end{pmatrix} \quad \begin{pmatrix} R \\ G \\ B \end{pmatrix} = \begin{pmatrix} 1.000 & 0.000 & 1.140 \\ 1.000 & -0.395 & -0.581 \\ 1.000 & 2.032 & 0.000 \end{pmatrix} \cdot \begin{pmatrix} Y \\ U \\ V \end{pmatrix}$$

$$Y = 0.299R + 0.587G + 0.114B, U = 0.492(B-Y), V = 0.877(R-Y)$$



L'occhio umano vede molto meglio la luminosità dei colori (è una questione fisica/biologica). Per cui si usa più informazione per la luminosità e meno per i colori.

Immaginiamo che lo spazio dei colori sia 3D. Nel caso dell'RGB "sbiotto" ho 3 assi perpendicolari con la stessa risoluzione. Nel caos YUV (e varianti) ho 3 assi sghembi e di cui uno ha una risoluzione maggiore (c'è molta approssimazione in questo parallelo).

Y luminosità

U e V delta tra colori e luminosità

Di fatto riesco a ricreare RGB. Molto usato per i video.



- Standardized variant of YUV
- Used for digital videos and some image format

$$\begin{aligned} Y &= w_R \cdot R + (1 - w_B - w_R) \cdot G + w_B \cdot B & R &= Y + \frac{1 - w_R}{0.5} \cdot C_r \\ C_b &= \frac{0.5}{1 - w_B} \cdot (B - Y) & G &= Y - \frac{w_B \cdot (1 - w_B) \cdot C_b - w_R \cdot (1 - w_R) \cdot C_r}{0.5 \cdot (1 - w_B - w_R)} \\ C_r &= \frac{0.5}{1 - w_R} \cdot (R - Y) & B &= Y + \frac{1 - w_B}{0.5} \cdot C_b \end{aligned}$$

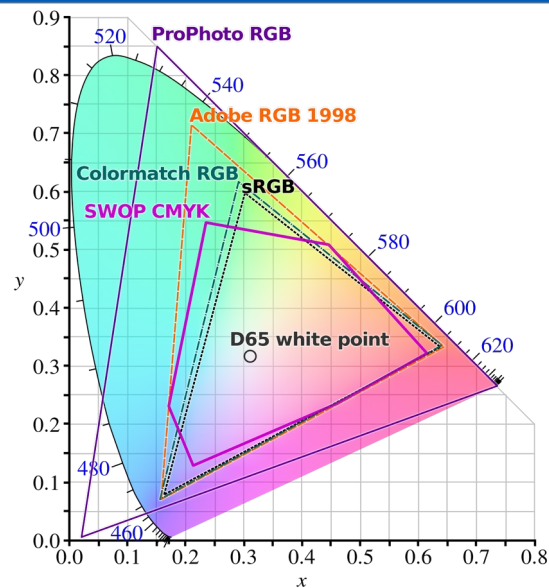
Cb blue difference

Cr red difference

I vari w_x sono fattori di scala la cui somma è 1

In pratica è una trasformazione di coordinate

- Subset of colors that can be represented



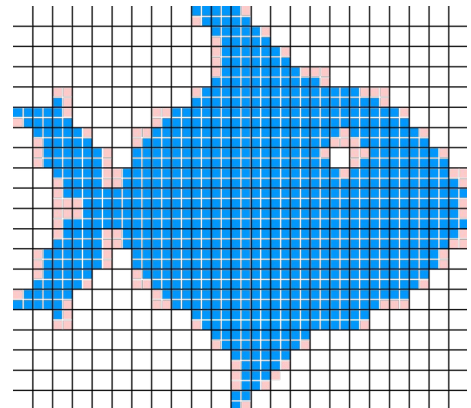
GAMUT → spazio dei colori rappresentabili

Y è la luminance

Le cifre in blu sono le lunghezze d'onda in nanometri



- For raster images, usually:
 - Header:
 - Format
 - Size
 - Color
 - Compression
 - ...
 - Bitmap
 - Pixel array(s)
- Every image standard features its own header/bitmap format



Abbiamo chiarito che informazione ci serve, come “codificarla” e come è organizzabile una immagine in “memoria” ovvero come una matrice.

Diverso è il concetto di “file” in cui ci vuole qualcosina di piú e in cui posso usare dei trucchetti anche per ridurre le dimensioni (le immagini hanno tanta ma tanta informazione).

Quindi se in memoria ha senso memorizzare l'immagine come matrice, su file le cose possono cambiare e di tanto.

- Portable Bit/Grey/Pix Map Format
 - .pbm, .pgm, .ppm
- Uncompressed format
- Very easy to parse
 - In some cases an ASCII editor can be used
- Born in the UNIX Environment
- ASCII Header
 - Format, Size, Comments
 - Comments widely used to encode additional info
- RAW or ASCII bitmap
- Video extension PVM

Lo presento pur essendo molto vecchio in quanto formato molto semplice. Scrivere da 0 codice che scrive/legge questi file è banale.

PNM Header & Bitmap

P4 = PBM
P5 = PGM
P6 = PPM

P5
320 240 ← Image Size
some free text ← Comments (add info)
255 ← Maximum channel value

(0,0)	(0,1)	...	(1,0)	(319,239)
-------	-------	-----	-------	-----------

PBM, PGM: 1 byte/pixel
PPM: 3 bytes/pixel

C'è un piccolo header ASCII che contiene le informazioni base del file come tipo (P4: bianco/nero, P5: toni di grigio, P6: colore), dimensioni matrice e massimo valore per luminosità o canale di colore RGB

Poi a seguire la matrice dei dati



- As memory bitmap

0	0	0	0	0	0	0	0
0	0	255	255	255	0	0	0
0	0	127	255	127	0	60	0
0	0	0	255	127	127	60	0
0	0	0	0	0	70	60	0
0	0	0	0	0	255	255	255

0	0	0	0	0	0	0	0
0	0	25	25	25	0	0	0
		5	5	5			
0	0	12	25	12	0	60	0
		7	5	7			
0	0	0	25	12	12	60	0
			5	7	7		
0	0	0	0	0	70	60	0
0	0	0	0	0	25	25	25
					5	5	5

- The more the information the bigger the memory required
- 4k images
 - $3840 \times 2160 \rightarrow 24 \text{ bit color} \rightarrow 23 \text{ Mbytes}$
- Size affects:
 - Memory (this can not be an issue for us!)
 - Storage
 - Transmission

PGM e PPM sono formati NON compressi. Se facciamo due conti vediamo che occupano un sacco di spazio!

I server di storage mica li regalano... Ma poi pensiamo anche a internet, quanto ci metto per scaricare una immagine di questo tipo?

E questo fa capire perché formati non compressi si usano molto raramente

- Compression can be mandatory
- Two different policies
- Lossless
 - Original info is preserved
 - Better choice for machine vision
 - TIFF, BMP, PNG, GIF, JPEG
- Lossy
 - Some info is lost
 - JPEG

Quindi i formati immagine usano in genere compressione dei dati. E anche qui ci sono due grandi famiglie, comprimo senza perdere informazioni (e quindi non tantissimo visto che parliamo di file binari con dati molto variabili) o rinuncio a qualcosa tipo dettagli minori (ma perdo quindi informazione)?/////

- Adobe Developers Association,
 - TIFF (TM) Revision 6.0 - Final, June 3, 1992
 - Compressed, lossless
 - Compression:
 - None, ZIP, LZW ...
 - Variable color depth
 - Multiple pages (Fax)

Non piú molto usato se non per documenti. Molto
ma molto versatile

- Device Independent Bitmap (DIB)
 - Typical extensions are .bmb or, less frequently, .dib
- File structure
 - BITMAPFILEHEADER bmfh;
 - type, size & layout (pixel ↔ lenght)
 - BITMAPINFOHEADER bmih;
 - size, compression and color format
 - RGBQUAD aColors[];
 - Color palette
 - Not present for 24 bit format (24-bit red-green-blue (RGB) for each pixel)
 - Colors are ordered on frequency basis (dithering)
 - BYTE aBitmapBits[];
 - Index or intensity of colors
 - Run-length encoded (RLE) compression

Qui si vede molto bene il concetto di header con tutte le informazioni sull'immagine. Può essere “palettizzato” (e quindi numero di colori limitato) o formato RGB

Compressione senza perdita tramite RLE (in pratica conto i pixel consecutivi dello stesso colore)

- Used for web and small animations
- Based on a color palette
 - 8 bit
 - Maximum 256 colors on 16 M max
 - Transparency support
- Patented compression algorithm
 - Big issue

Tuttora usato (per anni il problema del brevetto era stato un problema). Tipicamente immagini piccole, palette RGB con 24 bit per colore supporto trasparenza (rinunciando ad 1 colore su 256)

- PNG coped with patent issues
 - Portable Network Graphic
 - 24 bit depth (~16 millions of colors) + Alpha channel
- “Free” compression algorithm
- No animation support! → this explains why GIFs are still alive..
- 8 bytes initial header (magic number)
 - 89 50 4E 47 0D 0A 1A 0A (as ASCII first four are “.PNG”)
- Other “headers” and data are following as chunks
 - 4 bytes chunk length
 - 4 bytes chunk type (IHDR header, PLTE optional palette, IDAT actual data, IEND end of file)
 - Chunk data
 - 4 bytes CRC

Voleva superare i limiti e soprattutto il problema
Proprietà Intellettuale del formato GIF.



- JPEG: Joint Photographic Experts Group
- Born in the '80s (standard in '90s)
- JPEG is also a compression algorithm
- The idea was to have a platform independent (PC, Mac ...) file format
- Can use RGB, CMYK, YUV (maybe others)
- Magic number FF D8 FF

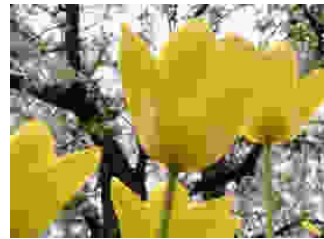
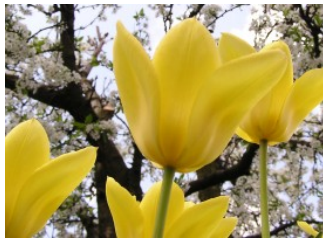
Formato tipicamente lossless

- Compression based on luminance and chrominance
- RGB o CMYK values are converted in $Y'C_bC_r$
 - Luminance + Chrominance
- Y and C_x are separately compressed
- For our eyes luminance is more important than colors
 - If a lossy compression is used, Luminance is better preserved
- Compression
 - Image is subdivided in 8x8 pixels blocks
 - Discrete Cosine Transform (~Fourier) is applied to each block
 - Results are quantized
 - Based on assumption the our eye is more sensitive to low frequency changes
 - Basically this wipes our small details

L'idea di base è trasformare l'immagine nello spazio frequenze e poi applicare un passa-basso ovvero tagliare le frequenze più alte e riconvertire nello spazio immagine. Dove si trovano le frequenze più alte? Dove l'immagine ha brusche variazioni → dettagli più fini

Per altro posso, variando la soglia del filtro, aumentare o diminuire compressione e salvaguardia dettagli

- Different compression levels can be used depending on image usage
 - Up to 20 → internet
 - Up to 5 → printed matter
- Good results with images without abrupt changes, namely “low frequency” images
- Bad if sharp color changes
- Avoid, if possible, for machine vision



Perdere dettagli significa comunque buttare via informazione, se possibile meglio non usarlo per visione artificiale

- Vector Graphics
- Described using a language!
 - Based on XML W3
- Different objects
 - Polylines, namely a combination of 1 or more straight and curve lines
 - This allows to obtain all shapes
 - Raster images
 - Text
 - ...

Esistono anche formati non raster. Non specifico esattamente pixel per pixel ma descrivo in qualche modo il contenuto (esempio: c'è una curva che inizia in (x_1, y_1) e va fino a (x_2, y_2) con curvatura r ecc. ecc.).

Vantaggio, posso scalare le dimensioni molto bene. Adatti per situazioni non complesse, possono contenere aree in cui inserisco una immagine in formato raster

SVG: example



```
<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<!-- Created with Inkscape (http://www.inkscape.org/) -->
<svg
  xmlns:dc="http://purl.org/dc/elements/1.1/"
  xmlns:cc="http://creativecommons.org/ns#"
  xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
  xmlns:svg="http://www.w3.org/2000/svg"
  xmlns="http://www.w3.org/2000/svg"
  xmlns:sodipodi="http://sodipodi.sourceforge.net/DTD/sodipodi-0.dtd"
  xmlns:inkscape="http://www.inkscape.org/namespaces/inkscape"
  width="1052.3622"
  height="744.09448"
  id="svg2"
  version="1.1"
  inkscape:version="0.48.1 r9760"
  sodipodi:docname="Nuovo documento 1">
  <defs
    id="defs4" />
  <sodipodi:namedview
    id="base"
    pagecolor="#ffffff"
    bordercolor="#666666"
    borderopacity="1.0"
    inkscape:pageopacity="0.0"
    ...
  inkscape:randomized="0"
    d="m 251.42858,175.52305 -61.11845,4.77394 -30.71632,53.05434
-23.42693,-56.65186 -59.949534,-12.81827 46.639804,-39.78672 -6.33453,-
60.976461 52.25191,32.062318 56.03459,-24.867261 -14.34635,59.602324 z"
    transform="translate(0,308.2677)"
    inkscape:transform-center-x="1.8913327"
    inkscape:transform-center-y="6.9973399" />
  </g>
</svg>
```



- Not images!
- Set of **data** points in the 3D space
- Free format
 - ASCII is often used
- We will use them sometime



Le “nuvole di punti” non sono propriamente immagini. Ma le useremo in qualche esercizio. Di fatto sono un elenco di punti in uno spazio 3D. Tipicamente per ogni punto avrò: le coordinate rispetto a un qualche sistema di riferimento, Altre informazioni quali colore del punto, luminosità, riflettività ecc. ecc.

Di solito generate o in seguito a elaborazione immagini o da dati provenienti da sensori differenti rispetto alle telecamere (LIDAR, RADAR...)

- Not always possible to choose
 - Avoid lossy formats
 - Limited depth formats
 - Quick & dirt approach → maybe you will have to purchase additional storage
- Not always necessary to choose
 - OpenCV
 - GOLD

Belli i formati immagine ma sebbene sia utile sapere cosa sono e cosa contengono di fatto i dettagli non ci interesseranno mai.

Tutti i SW utilizzati per lo sviluppo di sistemi di visione artificiale contengono già le varie primitive per leggere e scrivere detti file.

Vedremo che per OpenCV, che usiamo a lezione, basta usare una banale primitiva per leggere un qualunque tipo di file.

Analogo per chi farà tesi in VisLab in cui viene usato il SW GOLD.

- For processing it is “necessary” to load images
- Bidimensional array seems the straightforward choice
 - But monodimensional structures are widely used
- Carefully consider pixel data
- We will investigate the OpenCV approaches

Una volta letta l'immagine e caricata in memoria il formato originale non è significativo.

Di fatto avrò una matrice di pixel.

Verrebbe quindi da usare una matrice per la memorizzazione dell'immagine.

La maggioranza dei SW di elaborazione immagine però usano un'altra strategia: un vettore monodimensionale. In effetti nella maggioranza dei linguaggi di programmazione il manipolare matrici bidimensionali è semplice per accedere ai dati ma è in realtà molto complesso e poco efficiente quando voglio fare altre operazioni tipo ridimensionamenti, copie ecc.

L'uso di un vettore monodimensionale sebbene più scomodo per accedere ai singoli pixel presenta però vantaggi di gestione.

